

Relatório de Estratégia de Automação

Resumo

Este documento apresenta um direcionamento inicial para a estratégia de testes automatizados do projeto. Aborda: escopo (UI, API, Mobile), pirâmide de testes aplicada e mapeamento de prioridades e partes críticas.

1. Estratégia de Automação

Objetivo: agregar confiança rápida ao ciclo de desenvolvimento, reduzir regressões e acelerar entregas.

Abordagens:

- UI: automação de interface apenas para fluxos críticos de ponta a ponta (E2E) e validações visuais. Ex.: testes de fluxo de compra, login e recuperação de senha.
- API: foco principal; testar contratos, cenários felizes e casos de erro. Usar testes isolados (unit + integration) para validação de regras e contratos.
- Mobile: avaliar depois da estabilização do backend e UI web; priorizar testes em emuladores para fluxos críticos e smoke tests em dispositivos reais quando possível.

2. Pirâmide de Testes Aplicada

Base (Unit tests): 60% — testes rápidos, isolados, grandes volumes; cobrem lógica de negócio, helpers e utilitários.

Meio (Integration / API): 30% — testes que exercitam pontos de integração, contratos, persistência e validação de serviços.

Topo (E2E / UI): 10% — poucos testes E2E cobrindo fluxos críticos e regressões visuais.

Observações: Priorizar testes de API sobre UI para obter velocidade e confiabilidade. Usar mocks/stubs quando integrar depender de serviços instáveis ou custos altos.

3. Mapeamento de Prioridades e Partes Críticas

Alta Prioridade (automação imediata):

- Fluxos de autenticação (login, logout, recuperação de senha).
- Validações de entrada críticas (CPF, e-mail, regras de negócio principais).
- APIs de pagamento e checkout (se aplicável).
- Endpoints públicos e contratos (compatibilidade / regressão).

Média Prioridade: Fluxos de navegação principais do produto; Integrações com serviços terceirizados (notificações, emails).

Baixa Prioridade: Visualizações secundárias, páginas com baixo tráfego, experimentos A/B.

Recomendações de Arquitetura de Testes

Estrutura de pastas clara: tests/unit, tests/integration, tests/e2e.

Pipelines CI separados por tipo de teste (unit -> integration -> e2e). E2E em paralelo e condicional.

Test data e ambientes isolados: usar fixtures, factories e bancos temporários.

Ferramentas sugeridas: Unit: Jest; API: Supertest/axios + jest; E2E: Playwright ou Cypress.

Métricas e Critérios de Qualidade

Cobertura mínima orientada por risco: 80% unit nos módulos críticos.

Tempo médio de execução: manter os testes unitários < 2 minutos em CI; E2E isolados e limitados.

Falhas por bloco: rastrear tendência de falhas por suite (flaky detection).

Plano inicial de execução

1. Cobertura rápida de unit tests para utilitários e validações (utils/, src/scripts/validate*).
2. Automatizar testes API para endpoints expostos e casos de validação.
3. Criar 3-5 E2E cobrindo login, fluxo principal e validação de CPF/email.
4. Integrar execuções no CI em etapas e configurar alertas para regressões.

Documento gerado automaticamente: use o script `generate_report.py` para gerar o PDF `relatorio_estrategia.pdf`.